Lesson 8 — Server Virtualization (VMware) and Lab 01

INTE 22253 | Source: Lecture 08 "Server Virtualization" (45 slides) and Lab 01 "Creating a Virtual Machine, Installing LiteLinux 7.6, and Configuring a Static IP Address" | Builds on Lesson 5 (hypervisors, Type 1/2)

OBJECTIVES (SLIDE 3)

- Describe the need for virtualization on servers and identify the usage of server virtualization.
- Identify the **2 types of hypervisors** and the **3 types of virtualization** (full, para, hardware-assisted).
- Describe the advanced virtual machine operations used in server management (**vMotion, Storage vMotion**, DRS, HA, SRM).
- Understand how virtual machine **scalability** is achieved.
- Describe how networks in virtual servers are connected (**vNIC, vSwitch, uplinks**).
- Lab 01: create a VM on ESXi, install Linux and configure a static IP with `nmcli`.

1. Why virtualize servers? (slide 4)

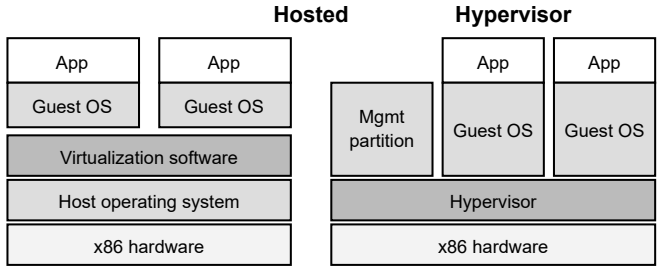
Reason	Slide points
Effective resource usage	RAM, CPU and disk can be allocated wherever needed.
Ease of management	Unused machines can be kept switched off indefinitely until required, consuming nothing but disk space. Easier application provisioning, maintenance, high availability and disaster recovery.
Security	Applications can be kept on separate (virtual) servers that do not pose a risk to each other — especially important when VMs are owned by **mutually untrusting users**.

2. Server virtualization scenarios (slides 5–6)

Hardware-based virtualization

Software-based virtualization

- Hosted (application virtualization)
- Hypervisor
  - Full virtualization (binary translation)
  - Para-virtualization (OS assisted)
  - Hardware-assisted virtualization (Intel VT-x / AMD-V)



Hosted = Type 2 (on a host OS) · Hypervisor = Type 1 (on hardware; a management partition/VM is used to administer it)

Figure 1 — Slides 5–6: the classification and the two architectures.

The three types of (hypervisor) virtualization

Background. A CPU has **privilege levels**. The OS kernel normally runs in the most privileged mode so it can use **privileged/sensitive instructions** (change memory mappings, talk to devices, disable interrupts). In a VM, the hypervisor must own the hardware, so the guest OS kernel cannot be allowed to run those instructions directly. Older x86 CPUs had some sensitive instructions that did not "trap" (alert the hypervisor) when run in a less privileged mode, which made x86 hard to virtualize. The three techniques solve this differently:

Type	How it works	Guest OS changed?	Slide notes and examples
Full virtualization (binary translation)	The hypervisor scans the guest kernel's code as it runs and **rewrites (translates)** problem instructions into safe sequences; ordinary user code runs directly on the CPU.	**No** — any unmodified OS runs	"Better performance; simple setup and installation." VMware ESXi, Microsoft Hyper-V, Xen (with AMD-SVM or Intel VT), VirtualIron (hardware-assisted)
Para-virtualization (OS assisted)	The guest kernel is **modified** to know it is virtualized; instead of privileged instructions it makes **hypercalls** (direct calls to the hypervisor).	**Yes** — needs a modified kernel (e.g. Linux)	"Best performance; ideal for multiple instances of host OS." Xen (with traditional hardware), Oracle VM Server
Hardware-assisted (Intel VT-x / AMD-V)	The CPU adds a special mode for guests; sensitive instructions automatically trap to the hypervisor, so no translation or kernel changes are needed.	**No**	"Same as full virtualization, however the guest VM can communicate directly with the hardware, bypassing the hypervisor for certain machine instructions." Needs explicit CPU support.

Application virtualization (slide 10, the hosted branch) "leverages operating system hardware qualification; manages hardware through the generic OS": an application is packaged to run isolated on top of a normal OS without being installed into it. Examples: Microsoft App-V, Citrix Virtual Apps. Slide 10's figure compares full virtualization (unmodified guests on hardware), para-virtualization (modified guests) and application implementation (virtualization application on a host OS).

3. Common hypervisors (slides 7–9)

Hypervisor	Slide description
Microsoft Hyper-V (Windows)	Native hypervisor included in Windows Server and Windows 10/11 Pro and Enterprise. Supports both Type 1 (bare-metal) and Type 2 (hosted) use. (When Hyper-V is enabled it actually runs beneath Windows, which becomes the "root partition".)
Xen (Linux, open source)	A Type 1 hypervisor that runs multiple OSes concurrently; supports **para-virtualization** (same hardware as host) and hardware virtualization.
KVM (Kernel-based Virtual Machine)	A Type 1 hypervisor **integrated into the Linux kernel**; converts the Linux kernel into a bare-metal hypervisor. Hardware-assisted, with emulated hardware for VMs.
LXC (Linux Containers)	Lightweight virtualization using the host's Linux kernel to run multiple isolated Linux systems (containers). **Not a traditional hypervisor**: OS-level virtualization.
VMware vSphere/ESXi (commercial)	Leading enterprise hypervisor for server virtualization: high availability, fault tolerance and vMotion.
Proxmox VE	Open-source virtualization management platform combining **KVM** (full virtualization) and **LXC** (containers).

4. Hardware-assisted virtualization (slides 11–16)

Full virtualization on x86 has significant costs in hypervisor complexity and run-time performance. Hardware extensions address the parts of x86 that are difficult or inefficient to virtualize, giving **simpler virtualization code and higher performance**, because instructions are handled in hardware instead of in software. There are three generations:

Area	Intel / AMD names	What it does
CPU (1st gen, 2006)	Intel **VT-x** (codename Vanderpool) / AMD **AMD-V** (also SVM, codename Pacifica)	Adds a guest mode so sensitive instructions trap to the hypervisor. Requires hardware **Data Execution Prevention (DEP)**: Intel calls it **Execute Disable (XD)**, AMD calls it **No Execute (NX)**; it stops code running from non-executable memory, blocking some buffer-overflow exploits. Must be enabled in the **BIOS**, and after enabling VT/AMD-V you must **POWER CYCLE** (switch fully off and on), **not just reboot**.
MMU (2nd gen)	Intel **EPT** (Extended Page Tables) / AMD **RVI** (Rapid Virtualization Indexing), also called **NPT** (Nested Page Tables)	Virtualizes the memory management unit in hardware, removing the need for **shadow page tables** and the overhead of keeping them updated. Gains: EPT up to 48% on MMU-intensive benchmarks (600% on microbenchmarks); RVI up to 42% (500%). Enable in BIOS.
I/O (newer)	Intel **VT-d** (Virtualization Technology for Directed I/O) / AMD **AMD-Vi** (IOMMU)	An **I/O memory management unit** remaps DMA transfers and device interrupts, so a VM can be given **direct access** to a hardware device such as a network card. VMware's **VMDirectPath I/O** uses it to let a VM use a physical NIC directly instead of an emulated or para-virtualized device: little throughput change, but lower CPU cost for network-heavy workloads.

SHADOW PAGE TABLES VS EPT/RVI (WHY THE MMU FEATURE MATTERS)

A guest OS keeps page tables mapping its *virtual* addresses to what it thinks are physical addresses ("guest physical"). Those are not real; the hypervisor must map guest physical → host physical. Without hardware help, the hypervisor builds **shadow page tables** (direct guest-virtual → host-physical maps) and must update them every time the guest changes its own tables: slow. With **EPT/RVI** the CPU walks both levels of tables itself (nested paging), so the hypervisor no longer maintains shadows.

Hyper-V high-level architecture (slide 13): the hypervisor runs on the hardware; a **root partition** (management OS, holds device drivers) controls it; **enlightened** Windows and Linux **child partitions** are para-virtualization aware and talk efficiently to the root; an **unenlightened** child partition runs an unmodified OS with emulated devices.

5. VMware products (slides 17–20)

	Workstation Player	Workstation Pro	vSphere Essentials Plus Kit	vSphere Standard	vSphere Foundation
Hypervisor type	Hosted	Hosted	Bare metal	Bare metal	Bare metal
Typical use	Test	Test & dev	Data centres and cloud infrastructure		
VMs per host	1–2	< 10	< 100		
Dedicated server / central & remote mgmt	No	No	Yes		
Licensing	Free for personal use	Perpetual	Subscription, core based (kit includes 96 core licences)	Subscription per core	

VMware vSphere Hypervisor (ESXi)

- Single-server partitioning
- Production-class hypervisor
- Advanced server resource management
- Free

VMware vSphere

- Pools of computing resources across many hosts
- Centralized management
- Built-in automation, availability and manageability
- All editions include ESXi

vSphere components (slides 19–20)

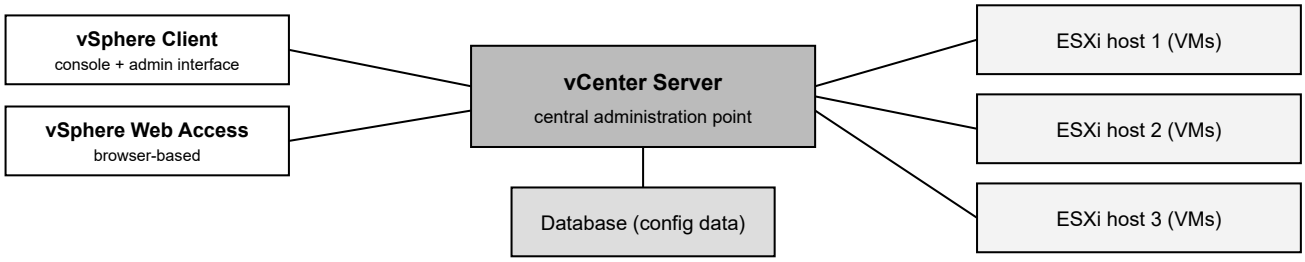


Figure 2 — Each vCenter Server manages many ESXi hosts; clients connect from workstations.

Component	Role
VMware ESXi	Virtualization layer that abstracts the host's processor, memory, storage and networking into multiple VMs
vCenter Server	Central administration point for ESX/ESXi hosts on a network; directs actions on VMs and hosts; the working core of vCenter
vCenter add-on modules	Extra capabilities: Update Manager, Converter, Guided Consolidation
vSphere Client	Installed on Windows; primary way to interact with vSphere: a console to operate VMs and an admin interface to vCenter and hosts
vSphere Web Access	Browser-based interface for remote access without the client
Databases	Organise all configuration data for the vSphere environment

6. vMotion and Storage vMotion (slides 21–25)

vMotion moves an **entire running VM from one host to another without downtime** — a **"hot" or "live" migration**. The VM's whole state is encapsulated, and the shared **VMFS** file system lets both source and target hosts access the VM's files at the same time, so only the **memory and execution state** travel, over a high-speed network. The VM keeps its network identity and connections.

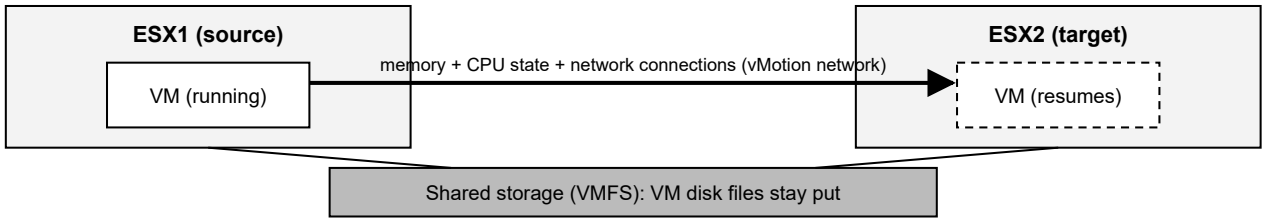


Figure 3 — vMotion moves the running state between hosts; the disks stay on shared storage.

vMotion operations, step by step (slide 23)

#	Step	Why
1	Migration request: move the VM from ESX1 to ESX2	Admin or DRS starts it
2	vCenter verifies the VM is in a stable state on ESX1 and that ESX2 is compatible (CPU, networking, etc.)	The VM must be able to run identically on ESX2
3	The VM is registered on ESX2	ESX2 prepares an empty VM shell
4	VM state (memory, registers, network connections) is copied to ESX2; changes made meanwhile are recorded in a **memory bitmap** on ESX1	The VM keeps running during the copy, so pages it changes ("dirty pages") are tracked
5	The VM is **quiesced** (paused very briefly) on ESX1 and the memory bitmap is copied to ESX2	Freeze so the last changes can be sent
6	The VM is started on ESX2; all requests now go to ESX2	Switch-over (users see no outage)
7	A final copy of the VM's memory is done from ESX1 to ESX2	Any remaining pages are fetched
8	The VM is unregistered from ESX1	Clean up the source
9	The VM resumes operation on ESX2	Migration complete

Storage vMotion (introduced in ESX 3.5) migrates a **running VM's disk files** from one **datastore** (a storage location) to another **on the same ESX host** (local or shared). Difference: **vMotion moves the VM to another host but keeps its storage location; Storage vMotion keeps the host but changes the storage location.**

#	Storage vMotion step (slide 25)
1	A new VM directory is created on the target datastore; configuration files and non-disk files are copied there
2	The ESX host does a **"self" vMotion** to the target directory
3	A **snapshot** (without memory) is taken of the VM's disks in the source directory (new writes go to the snapshot, freezing the base disk)
4	The VM disk files are copied to the target directory
5	The snapshot in the source directory is **consolidated** (merged) into the disk files in the target directory
6	Source disk files and directory are deleted

7. Management and scalability vServices (slides 30–37)

vService	What it gives
Physical-to-virtual migration	Consolidation management automates migrating physical machines into VMs (P2V)
Workflow for new VMs	Provisioning with approval processes (Life Cycle Manager, Stage Manager); self-provisioning (Lab Manager)
Scalability: vMotion	Move VMs without interrupting the applications inside
Automatic scalability: DRS	**Distributed Resource Scheduler** automatically balances workloads across hosts according to set limits and guarantees, removing the need to predict resource assignment (it uses vMotion to move VMs off busy hosts)
High availability: VMware HA	Protects all servers and applications against component and complete system failure (restarts VMs from a failed host on other hosts); one-click to configure
Disaster recovery: Site Recovery Manager (SRM)	Easy transition from a production site to a DR site; easy execution in a real disaster and easy testing ("for a good night's sleep")
Infrastructure vServices	VMware and Cisco collaborate on virtualization-aware networks (Nexus 1000V)
Security: VMSafe	Lets security vendors add security solutions inside the VMware infrastructure

8. Virtual networking (slides 27–29, 39–43)

8.1 ESX network components

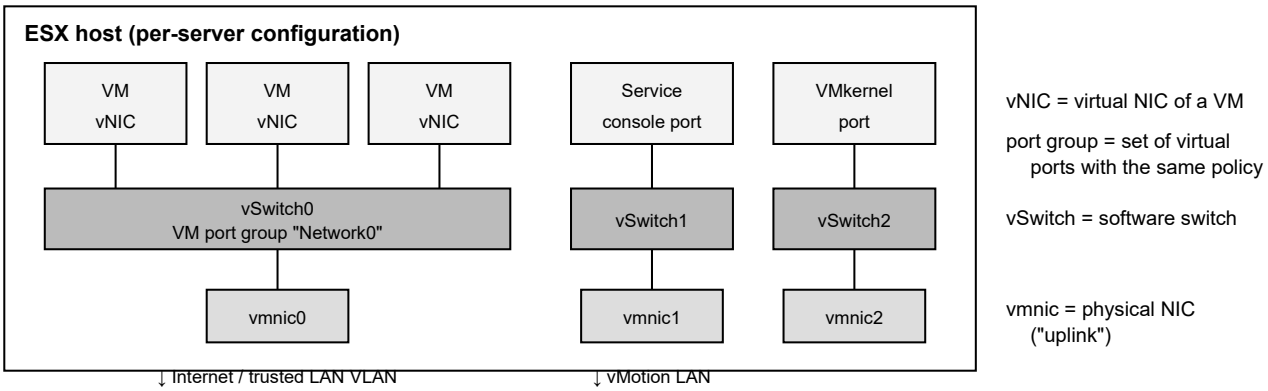


Figure 4 — Slides 39–41: VMs connect through vNICs to virtual ports on vSwitches; physical NICs (vmnics) are the uplinks.

- **ESX network architecture (slide 39):** an ESX host runs VMs (App + OS) alongside a **console OS**, on physical CPU, memory and network adapters, with three networks: **management network**, **VMkernel network** (used by the hypervisor itself, e.g. vMotion, storage traffic) and **production network** (VM traffic).
- **Slide 41 layout:** vSwitch0 has the virtual machine port group (Network0) with 10 VMs, uplink vmnic0 to the Internet/trusted LAN VLAN; vSwitch1 has the **service console port**, uplink vmnic1 to the vMotion LAN; vSwitch2 has a **VMkernel port**, uplink vmnic2. Separating traffic types on separate vSwitches and NICs keeps them from interfering.

8.2 vSwitch vs a physical switch (slides 42–43)

A **vSwitch** is a software implementation of an Ethernet switch inside ESX.

Like a physical switch	Different from a physical switch
Forwards frames by **MAC address**; supports **VLAN segmentation**	**No need to learn MAC addresses**: it already knows the MAC of every connected vNIC. **No participation in spanning tree** (STP: the protocol physical switches use to avoid loops). **No trunk between vSwitches**, so no loop is possible inside ESX unless a VM bridges two vSwitches.

Forwarding characteristics: forwarding is by MAC address with no learning; traffic that does not match a local VM's MAC is sent out to the vmnic (uplink); **VM-to-VM traffic on the same vSwitch stays local** (never touches the physical network); vSwitches **tag traffic with 802.1Q VLAN IDs** (802.1Q = the standard for adding a VLAN tag to Ethernet frames); vSwitches can create **EtherChannels** (several physical links bundled into one logical, faster, redundant link).

8.3 Cisco Nexus 1000V and successors (slides 27–29)

- **Nexus 1000V:** the industry's first third-party **virtual distributed switch** (one switch spanning many ESX hosts, managed like a Cisco switch), giving enhanced VM switching for VMware ESX. Features **VN-Link**: policy-based VM connectivity; **mobility of network and security properties** (the VM's port settings move with it); non-disruptive operational model; visibility and continued connectivity during vMotion.
- **VN-Link problems it solves:** vMotion may move VMs across physical ports, so network policy must follow; it was impossible to view or apply policy to locally switched (VM-to-VM) traffic; traffic from many VMs on one physical link could not be told apart. VN-Link extends the network to the VM, giving consistent services and coordinated, coherent management.
- **Newer options (slide 28): Cisco ACI** (Application Centric Infrastructure: an SDN solution with centralized, policy-driven automation and cloud integration); **Cisco Nexus 9000** (traditional and programmable switches, supports ACI); **VMware NSX** (network virtualization platform that builds entire networks in software, with **micro-segmentation**: firewall rules per VM); **Cisco Meraki** (cloud-managed networking).

9. Lab 01 — Create a VM, install LiteLinux, configure a static IP

Objectives: connect to VMware ESXi · create a VM · install LiteLinux 7 · configure a static IP · verify connectivity.
Requirements: VMware ESXi, vSphere Client, LiteLinux ISO, 2 vCPU, 4 GB RAM, 40 GB disk.

#	Lab step	What it means
1	Log in to the ESXi web client	Open the ESXi host's address in a browser; ESXi is a Type 1 hypervisor
2–3	Create/Register VM; name `LiteLinux-Lab01`	Start the new-VM wizard
4	Guest OS: Linux → RHEL 8 or Other Linux (64-bit)	Tells ESXi which virtual hardware and drivers suit the guest
5	Select datastore	Where the VM's files (virtual disk, config) will live
6	2 vCPU, 4 GB RAM, 40 GB disk, VM Network, attach the LiteLinux ISO	Allocate virtual resources (Lesson 5: CPU time-slicing, mapped RAM, virtual disk file); connect the vNIC to the "VM Network" port group; the ISO acts as a virtual DVD to install from
7–8	Power on, install LiteLinux, create a root password and a student user	Normal OS installation inside the VM
9	`nmcli device status`	NetworkManager CLI: list network devices and find the interface name (here `ens160`, the VMware virtual NIC)
10	Configure the static IP (below)	Fix the address instead of getting one from DHCP
11	`sudo nmcli connection down ens160` then `up ens160`	Restart the connection so the new settings apply
12	`ip addr`, `ip route`, `ping -c 4 8.8.8.8`, `ping -c 4 www.google.com`	Verify (table below)

```
sudo nmcli connection modify ens160 \
     ipv4.addresses 192.168.8.X/24 \
     ipv4.gateway 192.168.8.1 \
     ipv4.dns "8.8.8.8 1.1.1.1" \
     ipv4.method manual
```

Part	Meaning
`ipv4.addresses 192.168.8.X/24`	The VM's own IP. **/24** = subnet mask 255.255.255.0: the first 24 bits (192.168.8) are the network, the last number X (1–254) is the host. Each student uses a different X. (The lab sheet prints "192.1681. X/24", a typo; it must be in the same network as the gateway 192.168.8.1.)
`ipv4.gateway 192.168.8.1`	The router used to reach other networks/the Internet
`ipv4.dns "8.8.8.8 1.1.1.1"`	DNS servers (Google, Cloudflare) that turn names like www.google.com into IP addresses
`ipv4.method manual`	Use these static settings instead of DHCP (automatic)

Verification	Expected result	If it fails
`ip addr`	ens160 shows `inet 192.168.8.X/24`	Address not applied: re-check modify, restart connection
`ip route`	`default via 192.168.8.1 dev ens160`	Gateway missing or wrong
`ping -c 4 8.8.8.8`	4 replies (Internet reachable by IP)	Gateway/VM network/port group problem
`ping -c 4 www.google.com`	Name resolves and 4 replies	If IP ping works but this fails: **DNS** problem

Why a static IP for servers? Clients and other servers must always find the server at the same address; a DHCP address could change after a reboot.

10. Summary (slide 44)

Roles of server virtualization in an enterprise · role of the hypervisor · types of hypervisors (Type 1/2) · types of virtualization (full, para, hardware-assisted) · management techniques for virtual servers (vCenter, vMotion, Storage vMotion, DRS, HA, SRM) · achieving scalability (vMotion, DRS) · networking for virtual servers (vNIC, vSwitch, port groups, uplinks, Nexus 1000V).

11. Exam-style questions with model answers

Q1. Give three reasons for virtualizing servers. (3 marks)
Effective resource usage (CPU, RAM, disk allocated where needed); ease of management (unused VMs kept off, easier provisioning, maintenance, HA and DR); security (applications isolated on separate VMs that do not risk each other, even for mutually untrusting users).

Q2. Compare full virtualization, para-virtualization and hardware-assisted virtualization. (9)
Full virtualization uses binary translation: the hypervisor rewrites sensitive guest kernel instructions on the fly, so unmodified OSes run (VMware ESXi, Hyper-V); simple setup but translation costs performance. Para-virtualization modifies the guest kernel to call the hypervisor through hypercalls instead of privileged instructions; best performance but needs a modified OS (Xen on traditional hardware, Oracle VM Server). Hardware-assisted virtualization uses CPU extensions (Intel VT-x, AMD-V) so sensitive instructions trap to the hypervisor in hardware and the guest can execute certain instructions directly; unmodified guests, simpler hypervisor code and higher performance, but needs CPU support enabled in the BIOS.

Q3. Explain the three generations of hardware-assisted virtualization. (6)
CPU: Intel VT-x / AMD-V (2006) add a guest mode; need DEP (Intel XD, AMD NX) enabled in BIOS and a power cycle. MMU: Intel EPT / AMD RVI (NPT) do two-level address translation in hardware, removing shadow page tables (up to 48%/42% gains on MMU-heavy benchmarks). I/O: Intel VT-d / AMD-Vi IOMMU remaps DMA and interrupts so VMs can access devices such as NICs directly (VMware VMDirectPath I/O), lowering CPU cost.

Q4. Distinguish Xen, KVM and LXC. (6)
Xen: Type 1 hypervisor running multiple OSes, supporting para-virtualization and hardware virtualization. KVM: Type 1 hypervisor built into the Linux kernel, turning Linux into a bare-metal hypervisor using hardware assistance. LXC: not a hypervisor but OS-level virtualization, running isolated Linux containers that share the host kernel.

Q5. Compare ESXi with vSphere and describe vCenter Server. (5)
ESXi is the free, production-class bare-metal hypervisor for partitioning a single server with advanced resource management. vSphere is the suite that pools many ESXi hosts into resources with centralized management, automation and availability; all editions include ESXi. vCenter Server is the central administration point that manages multiple ESXi hosts and directs actions on VMs and hosts, with a database for configuration and the vSphere Client/Web Access as interfaces.

Q6. What is vMotion? Describe its steps. (9)
vMotion is live migration of a running VM between hosts without downtime; shared VMFS storage lets both hosts access the VM's files, so memory and execution state are sent over a fast network and the VM keeps its network identity. Steps: request ESX1 → ESX2; vCenter checks the VM is stable and ESX2 is compatible; VM registered on ESX2; memory, registers and network state copied while changes are tracked in a memory bitmap on ESX1; VM quiesced on ESX1 and the bitmap copied; VM started on ESX2 and requests redirected; final memory copy; VM unregistered from ESX1; VM resumes on ESX2.

Q7. Distinguish vMotion and Storage vMotion. Outline Storage vMotion. (6)
vMotion moves a running VM to another host while its storage stays put; Storage vMotion keeps the VM on the same host but moves its disk files to another datastore. Storage vMotion: create the target directory and copy config files; self-vMotion to the target directory; snapshot the source disks; copy the disks; consolidate the snapshot into the target disks; delete the source files.

Q8. Explain DRS, VMware HA and Site Recovery Manager. (6)
DRS (Distributed Resource Scheduler) automatically balances workloads across hosts within set limits and guarantees, using vMotion, so resource assignment need not be predicted. VMware HA protects servers and applications against component and whole-host failure by restarting affected VMs on other hosts. SRM automates failover from the production site to a disaster-recovery site and allows non-disruptive DR testing.

Q9. How does a vSwitch differ from a physical switch? (6)
Like a physical switch, it forwards by MAC address and supports VLANs (802.1Q tagging) and EtherChannels. Unlike one, it does not need to learn MAC addresses (it knows each connected vNIC's MAC), does not take part in spanning tree, and has no trunks between vSwitches, so loops are impossible without a bridging VM. VM-to-VM traffic on the same vSwitch stays local; unknown destinations go out through the vmnic uplink.

Q10. What problems does Cisco Nexus 1000V/VN-Link solve? (4)
When vMotion moves VMs across physical ports, network and security policy must follow the VM; locally switched VM-to-VM traffic could not be seen or policed; traffic from many VMs on one link could not be distinguished. Nexus 1000V is a distributed virtual switch whose VN-Link extends the network to each VM, with policy-based connectivity that moves with the VM and visibility during vMotion.

Q11. In Lab 01, explain each part of the nmcli command that sets a static IP and how you verify it. (6)
`nmcli connection modify ens160` edits the ens160 connection; `ipv4.addresses 192.168.8.X/24` sets the host address with a 255.255.255.0 mask; `ipv4.gateway 192.168.8.1` sets the default router; `ipv4.dns "8.8.8.8 1.1.1.1"` sets DNS servers; `ipv4.method manual` switches from DHCP to static. Restart with `nmcli connection down/up ens160`. Verify with `ip addr` (address), `ip route` (default via 192.168.8.1), `ping 8.8.8.8` (Internet by IP) and `ping www.google.com` (DNS working).

12. Common mistakes

Mistake	Correct idea
Para-virtualization runs unmodified OSes	Para-virtualization needs a modified guest kernel (hypercalls); full and hardware-assisted run unmodified OSes.
Binary translation is done by the CPU	The hypervisor (software) translates; hardware-assisted is the CPU-based approach.
Reboot after enabling VT-x	Slide: you must POWER CYCLE, not just reboot.
EPT is for I/O devices	EPT/RVI = MMU (memory); VT-d/AMD-Vi = I/O.
LXC is a hypervisor	LXC is OS-level (container) virtualization.
vMotion moves the VM's disks	vMotion moves host (state); Storage vMotion moves disks between datastores.
vMotion needs downtime	It is a live ("hot") migration with no downtime.
vSwitches learn MACs and run spanning tree	They know vNIC MACs already and do not run STP.
vmnic = virtual NIC	vmnic = physical NIC (uplink); vNIC = the VM's virtual NIC.
Using gateway 192.168.8.1 with IP 192.168.1.X/24	The IP must be in the gateway's network: 192.168.8.X/24.

Cheat sheet

Concepts	
Why	Resource usage, ease of management, security (untrusting users)
Hosted	App virtualization on host OS (Type 2); App-V, Citrix Virtual Apps
Full	Binary translation; unmodified guest; ESXi, Hyper-V
Para	Modified guest, hypercalls; best perf; Xen, Oracle VM
HW-assisted	VT-x / AMD-V; guest runs some instructions directly
CPU	VT-x (Vanderpool) / AMD-V (SVM, Pacifica); DEP: XD / NX; BIOS; **power cycle**
MMU	EPT / RVI (NPT): no shadow page tables; +48% / +42%
I/O	VT-d / AMD-Vi (IOMMU); VMDirectPath I/O
Hypervisors	Hyper-V (Type 1+2), Xen (para), KVM (in Linux kernel), LXC (containers), ESXi, Proxmox (KVM + LXC)
Hyper-V	Root partition + enlightened / unenlightened child partitions

Lab 01	
VM	ESXi web client → create VM LiteLinux-Lab01, RHEL 8/Other Linux 64-bit, 2 vCPU, 4 GB, 40 GB, VM Network, ISO
Find NIC	`nmcli device status` → ens160
Static IP	`nmcli connection modify ens160 ipv4.addresses 192.168.8.X/24 ipv4.gateway 192.168.8.1 ipv4.dns "8.8.8.8 1.1.1.1" ipv4.method manual`
Apply	`nmcli connection down/up ens160`
Verify	`ip addr`, `ip route`, `ping -c 4 8.8.8.8`, `ping -c 4 www.google.com`

VMware	
Type 2	Workstation Player (1–2 VMs, free), Pro (<10, perpetual)
Type 1	vSphere Essentials Plus / Standard / Foundation (<100 VMs/host, per-core subscription)
ESXi	Single server, free, production-class
vSphere	Resource pools, central mgmt, automation, includes ESXi
Components	ESXi, vCenter Server, modules, vSphere Client, Web Access, databases
vMotion	Live host migration, shared VMFS. Request → check → register ESX2 → copy state + bitmap → quiesce, copy bitmap → start ESX2 → final copy → unregister ESX1 → resume
Storage vMotion	Same host, new datastore: dir + config → self-vMotion → snapshot → copy disks → consolidate → delete source
vServices	P2V, Lab/Stage Manager, vMotion, DRS (balance), HA (restart), SRM (DR), Nexus 1000V, VMSafe

Networking	
Path	VM vNIC → port group → vSwitch → vmnic uplink
Networks	Management, VMkernel (vMotion/storage), production
vSwitch	MAC forwarding (no learning), VLAN 802.1Q, EtherChannel; no STP; no trunks; VM-VM local
Nexus 1000V	3rd-party distributed vSwitch; VN-Link: policy follows VM
Newer	Cisco ACI (SDN), Nexus 9000, VMware NSX (micro-segmentation), Meraki (cloud-managed)